



INTRODUCTION



Hundreds of millions of devices use the Microsoft Windows platform. Many of the world's largest companies rely on its security to protect their data and communications, as does anyone hosting their code in the Azure cloud. But because Windows is so important to the security of the modern internet, it's also a popular target for attack.

The Windows NT operating system began including security in its design in 1993, when it introduced user accounts, control over resources, and remote access from a network. In the more than 20 years since then, much has changed in Windows security. Microsoft has replaced its original authentication process with modern technology, granted the access control mechanism additional capabilities, and significantly hardened the platform against attack.

Today, the security of the Windows platform is surprisingly complex, and many attacks rely on abusing this complexity. Unfortunately, Microsoft's documentation in this area can be lacking. As Windows is not open source, sometimes the only way to understand its security is through deep research and analysis.

This is where I come in. I've spent more than 20 years as a developer and security researcher on Windows platforms, cultivating an understanding of the operating system's undocumented corners. In this book, I share some of my extensive expertise in an easy-to-understand form. By mastering the principles of Windows security, you'll be able

to kick-start your own research project or improve your software product.

Who Is This Book For?

I wrote this book for people who work with Windows security. Perhaps you're a developer of Windows software and want to ensure that your product is secure. Or maybe you're a system administrator tasked with securing Windows across an enterprise and don't fully understand how various security features combine to protect the platform. Or you might want to poke holes in the operating system to find security vulnerabilities as a researcher.

This book assumes reasonable familiarity with the Windows user interface and its basic operations, such as manipulating files. That said, you don't need to be a low-level Windows expert: for those who need a little more grounding, [Chapters 2](#) and [3](#) provide an overview of the operating system and how it's put together.

I rely heavily on the use of PowerShell scripting, so you'll find it helpful to have some experience with the language, as well as with the .NET framework on which it's based. To get you up to speed, [Chapter 1](#) gives a very quick overview of some of PowerShell's features. Elsewhere, I'll do my best to avoid using esoteric features of the language, to keep the code accessible to readers with knowledge of other scripting languages or shell environments (such as bash).

What Is in This Book?

In each chapter, we'll cover core security features implemented in modern versions of Windows. We'll also walk through several worked examples written in PowerShell, which should give you a better understanding of the commands introduced in the chapter. Here's a brief summary of what each chapter covers.

Part I surveys the Windows operating system from a programming perspective. It should provide you with the foundation needed to understand the material in the rest of the book.

Chapter 1: Setting Up a PowerShell Testing Environment In this chapter, you'll set up PowerShell to run the examples included in the subsequent chapters. This includes installing a PowerShell module I've written to interact with Windows and its security features. The chapter also provides an overview of the PowerShell scripting language.

Chapter 2: The Windows Kernel This chapter covers the basics of the Windows kernel and its system call interface, a topic crucial to developing a solid understanding of Windows security. I also describe the object manager, used to manage resources.

Chapter 3: User-Mode Applications Most applications don't directly use the system call interface from the kernel; instead, they use a set of higher-level programming interfaces. This chapter covers Windows features such as file handling and the registry.

Part II covers the most important component of the Windows kernel for security, the Security Reference Monitor. We'll look at all aspects of access control, from constructing the user's identity to securing an individual resource, such as a file.

Chapter 4: Security Access Tokens Windows assigns every running process an access token, which represents the user's identity to the system. This chapter describes the various components stored in the token that are used to check access.

Chapter 5: Security Descriptors Each securable resource needs a description of who is allowed to access it and what type of access they are granted. This is the purpose of security descriptors. In this chapter, we'll cover their internal structure and how you can create and manipulate them.

Chapter 6: Reading and Assigning Security Descriptors To inspect the security of the system, you need to be able to query the security descriptor of a resource. This chapter explains how this querying happens for different types of resources. It also covers the many complex ways that Windows assigns security descriptors to resources.

Chapter 7: The Access Check Process Windows uses the access check to determine what access to grant a user to a resource. This operation takes the token and the security descriptor and follows an algorithm to determine the granted access. This chapter works through a PowerShell implementation of the algorithm to explore its design in depth.

Chapter 8: Other Access Checking Use Cases Although Windows primarily uses access checks to grant access to resources, it sometimes uses them to determine other security properties, such as the visibility of resources and whether a process is running with a low level of privilege. This chapter covers these alternative use cases for the access check.

Chapter 9: Security Auditing The access check process can also create logs of the resources a user has accessed, and with what level of access. This chapter covers these system auditing policies.

Part III contains details of Windows authentication, the mechanisms that verify a user's identity for the purposes of access control.

Chapter 10: Windows Authentication As the topic of authentication is quite complex, this chapter summarizes the authentication structure and services on which the rest of the authentication mechanisms depend.

Chapter 11: Active Directory Windows 2000 introduced a new model for networking Windows systems in an enterprise, with all authentication information stored in a network directory that users and

administrators could query and modify. This chapter covers how Active Directory stores information and secures it from malicious modification.

Chapter 12: Interactive Authentication The most common authentication scenario on Windows occurs when a user enters their username and password into their computer and gains access to the desktop. This chapter covers how the operating system implements this authentication process.

Chapter 13: Network Authentication When a user wants to access a network service in a Windows enterprise network, they typically must authenticate to it. Windows provides special network protocols to implement this authentication without disclosing the user's credentials to a potentially hostile network. This chapter explains the network authentication process, focusing on the New Technology LAN Manager (NTLM) authentication protocol.

Chapter 14: Kerberos Along with Active Directory, Windows 2000 also introduced the use of the open Kerberos authentication protocol for enterprise network authentication. This chapter explains how Kerberos works in Windows to authenticate a user interactively and over a network.

Chapter 15: Negotiate Authentication and Other Security

Packages Over the years, Windows has added other types of network authentication protocols. This chapter covers these new types, including Negotiate, to supplement those discussed in **Chapters 13** and **14**.

Finally, the two appendices provide configuration details and further resources.

Appendix A: Building a Windows Domain Network for Testing To run some of the examples in the book, you'll need a Windows domain

network. This appendix provides some steps for using PowerShell to configure a network for testing.

Appendix B: SDDL SID Alias Mapping This appendix provides a table of constants referenced in **Chapter 5**.

PowerShell Conventions Used in This Book

The PowerShell scripting language, which is included with all versions of Windows, is one of the best ways to flexibly experiment with the internals of the operating system without needing to install much additional software. As PowerShell is based on the .NET runtime, this book will use a .NET library I've written for interacting with Windows, making it easy to develop complex scripts. All example scripts in the book will be available to download from <https://github.com/tyranid/windows-security-internals>.

The PowerShell examples in each chapter follow a common set of style conventions that should help you understand how to use them. Each example is provided as a listing, of which there are two types: interactive and non-interactive. Interactive PowerShell listings are those you should enter on the command line to observe the results. Here is an example of an interactive listing:

```
❶ PS> ls C:\
❷ Directory: C:\
    Mode                LastWriteTime         Length Name
    ----                -
    d-r--                4/17  11:45 AM         Program Files
❸ --snip--
```

An interactive listing precedes each command to enter with a PowerShell-style prompt (PS>) and shows the command in bold ❶. You'll see the resulting output below the command ❷. Sometimes the output can be quite long, so to save space, I use `--snip--` to indicate that

the output has been truncated ❸. Also note that in some examples the output is indicative; it might be subtly different depending on your operating system or network configuration.

Most of the interactive listings are designed to be executed from a normal user account. However, some must run under an administrator account to access certain protected features. If you don't run the commands as an administrator, the results won't be correct. The text preceding each listing will clarify whether you must run the command as an administrator.

A non-interactive listing contains PowerShell code that you can copy into a script file for reuse, like this:

```
function Get-Hello {  
    "Hello"  
}
```

Non-interactive listings don't include the PowerShell prompt and aren't in bold.

If you've written any scripts in PowerShell, you'll know that the language is notorious for verbose command and parameter names. This makes it difficult to fit certain commands on a single line in the book. Here is an example of a long PowerShell line and a few ways the book might split it to make it fit on the page:

```
PS> Get-ChildItem -LiteralPath "C:\" -Filter "*.exe" -Recurse -Hidden  
❶ -System -Depth 5 | Where-Object {  
    ❷ $_.Name -eq "Hello"  
}
```

The first line, using the `Get-ChildItem` command, is too long to fit on the page, so it wraps onto a subsequent line ❶. You can't just add a new-

line in the middle of such a command, so when you're entering it into the shell or a file, you should treat it as a single line. The key indicator that the line continues, instead of being part of the output, is that there's a bold character in the first column.

PowerShell can break long lines on certain characters, such as the pipe (`|`), the comma (`,`), or braces (`{}`). In this listing, I've added a new-line following the opening brace (`{`) and placed the subsequent commands in the braced block, indented one level ❷. In this case, the shell will handle the introduction of the new line. Note that the closing brace (`}`) is in the first column, so you might assume it needs to be placed on the previous line. While moving the brace to the previous line will still work in this specific case, it's unnecessary.

Note that the Windows operating system is still under active development. While all the PowerShell examples have been tested on the latest versions of Windows available at the time of writing, there is a chance that new security features will have been introduced, or older ones deprecated, by the time you come to read this book. The following is a list of the versions on which the examples were tested, along with the major OS build number:

- Windows 11 (OS build 22H2)
- Windows 10 (OS build 19H2)
- Windows Server 2022 (OS build 22H2)
- Windows Server 2019 (OS build 18H2)

Any mentions of “the latest versions” in the text refer to these versions.

Getting in Touch

I'm always interested in receiving feedback, both positive and negative, on my work, and this book is no exception. You can email me at winsecinternals.book@gmail.com. You can also subscribe to my blog at

<https://www.tiraniddo.dev>, where I post some of my latest advanced security research.